Listing 2.4. Detaching a thread to handle other documents

```
void edit_document(std::string const& filename)
{
    open_document_and_display_gui(filename);
    while(!done_editing())
    {
        user_command cmd=get_user_input();
        if(cmd.type==open_new_document)
        {
            std::string const new_name=get_filename_from_user();
            std::thread t(edit_document,new_name);                    <-(1)
            t.detach();                                               <-(2)
        }
        else
        {
            process_user_input(cmd);
        }
    }
}
```

If the user chooses to open a new document, you prompt them for the document to open, start a new thread to open that document (1), and then detach it (2). Because the new thread is doing the same operation as the current thread but on a different file, you can reuse the same function (`edit_document`) with the newly chosen filename as the supplied argument.

This example also shows a case where it's helpful to pass arguments to the function used to start a thread: rather than just passing the name of the function to the `std::thread` constructor (1), you also pass in the filename parameter. Although other mechanisms could be used to do this, such as using a function object with member data instead of an ordinary function with parameters, the Thread Library provides you with an easy way of doing it.

2.2. Passing arguments to a thread function

As shown in listing 2.4, passing arguments to the callable object or function is fundamentally as simple as passing additional arguments to the